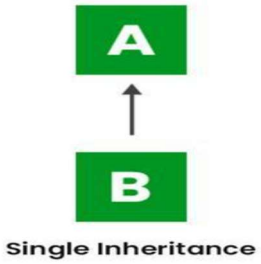
Explain the concept of inheritance in programming.

Inheritance is a concept that allows a child class to acquire the properties and behavior of a parent class.
To inherit a class, you simply incorporate the definition of one class into another by using the extend keyword.
It promotes code reusability, method overriding, polymorphism.
Super class: the class whose features are inherited.
Sub Class: the class that inherits from another class.
Syntax:

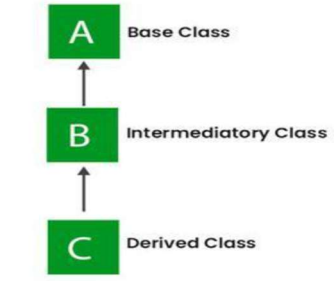
```
class parentclass{
//body
}
class childclass extends parentclass{
//body
}
```

Types of inheritance:
1. Single inheritance: In single inheritance, a sub-class is derived from only one super class. It inherits the properties and behavior of a single-parent class.
'A' is a parent class and 'B' is a child class. The class 'B' inherits all the properties of the class 'A'.



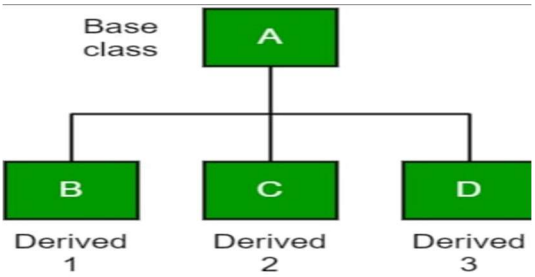
Single Inheritance

2. Multilevel inheritance: a derived class will be inheriting a base class, and as well as the derived class also acts as the base class for other classes.
class A serves as a base class for the derived class B, which in turn serves as a base class for the derived class C.



Multilevel Inheritance

3. Hierarchical inheritance: one class serves as a superclass (base class) for more than one subclass.
class A serves as a base class for the derived classes B, C, and D.



Example Program:

```
class A {
int i;
int j;
void setij(int x, int y) {
i = x;
j = y;
}
}
class B extends A {
int total;
void sum() {
total = i + j;
}
}

class Access {
public static void main(String args[]) {
B subOb = new B();
subOb.setij(10, 12);
subOb.sum();
System.out.println("Total is " + subOb.total);
}
}
```

```
Total is 22
```

Illustrate multilevel hierarchy using an example.

It is a form of inheritance where a class inherits from another class, which in turn inherits from another class, creating a chain of inheritance.

Multiple levels of parent-child relationships are allowed.

Each subclass inherits properties and methods from all its superclass in the hierarchy.

```
class GrandParent {
    void showGrandParent() {
        System.out.println("This is Grandparent class.");
    }
}
class Parent extends GrandParent {
    void showParent() {
        System.out.println("This is Parent class.");
    }
}
class Child extends Parent {
    void showChild() {
        System.out.println("This is Child class.");
    }
}
public class Multilevel {
    public static void main(String[] args) {
        Child child = new Child();
        child.showGrandParent(); // Inherited from GrandParent
        child.showParent();      // Inherited from Parent
        child.showChild();       // Child's own method
    }
}
```

```
This is Grandparent class.
This is Parent class.
This is Child class.
```

Demonstrate two uses of super keyword with example

1. Using super to Call Superclass Constructors

A subclass can call a constructor defined by its superclass by use of the following form of **super**:
super(*arg-list*);
Here, *arg-list* specifies any arguments needed by the constructor in the superclass. **super( )** must always be the first statement executed inside a subclass' constructor.
To see how **super( )** is used, consider this improved version of the **BoxWeight( )** class:

```
class BoxWeight extends Box {
double weight;
BoxWeight(double w, double h, double d, double m) {
super(w, h, d);
weight = m;
}
}
```

The second form of **super** acts somewhat like **this**, except that it always refers to the superclass of the subclass in which it is used. This usage has the following general form:

super.*member*

Here, *member* can be either a method or an instance variable.
This second form of **super** is most applicable to situations in which member names of a subclass hide members by the same name in the superclass. Consider this simple class hierarchy:

```
class A {
int i;
}
class B extends A {
int i;
B(int a, int b) {
super.i = a;
i = b;
}
void show() {
System.out.println("i in superclass: " + super.i);
System.out.println("i in subclass: " + i);
} }
class UseSuper {
public static void main(String args[]) {
B subOb = new B(1, 2);
subOb.show();
}
}
```

```
i in superclass: 1
i in subclass: 2
```